

Cloud Computing and Virtualization

Bachelors in Computer Engineering

**Prepared by Er. Ganga Gautam,
Assistant Professor, Pokhara University**

Chapter 4

Resource Management in Cloud Computing(5 hrs)

Outlines:

1. Scaling
 - 1.1. Scaling Strategies
 - 1.2. Types
 - 1.3. Load Balancing
2. Redundancy and High Availability
3. Cloud monitor mechanisms
 - 3.1. Cloud Usage Monitor, Automated Scaling Listener, Load Balancing, Audit Monitor, Pay-Per-Use Monitor
4. Service Level Agreement and its Types
5. Billing Management System in cloud computing

Resource Management

- **Resource Management** = the process of efficiently allocating and controlling compute, storage, and network resources.
- Ensures that every application receives the required performance without overusing cloud infrastructure.
- **Dynamic provisioning:** Resources are added or released based on demand in real time.
- **Key Goal:** Balance between performance, availability, and cost efficiency.
- **Examples:**
 - AWS Auto Scaling: Automatically adjusts instance capacity.
 - Azure Resource Manager (ARM): Manages infrastructure deployment and access control.

Why Resource Management Matters

- Prevents over-provisioning (wasting idle resources).
- Prevents under-provisioning (performance degradation).
- Ensures SLA compliance (availability & performance guarantees).
- Reduces operational costs through optimization.
- Enables scalability and flexibility for variable workloads.

Policies	
Admission control.	The explicit goal of an admission control policy is to prevent the system from accepting workloads in violation of high-level system policies
Capacity allocation.	To allocate resources for individual instances; an instance is an activation of a service.
Load balancing.	Distribute the workload evenly among the servers.
Energy optimization.	Minimization of energy consumption.
Quality-of-service (QoS) guarantees.	Ability to satisfy timing or other conditions specified by a Service Level Agreement.

There are various ways to implement resource management policies:

- **Control-theory based** — using feedback loops: monitor resource usage, adjust resources based on measured load and predicted demand for stability.
- **Machine-learning based** — using data-driven / predictive models instead of fixed performance models. Good for dynamic and unpredictable workloads.
- **Utility-based / economic / market-oriented methods** — treat resources as commodities; allocate based on cost, demand, “bidding” for bundles of resources (e.g. CPU + memory + bandwidth) rather than simple static allocation.

Resource Lifecycle in Cloud

The **Resource Lifecycle** typically includes:

1. Request: User or system requests additional compute/storage/network.
2. Allocation: Resources provisioned from the cloud pool.
3. Monitoring: Continuous performance and utilization tracking.
4. Scaling: Automatic adjustments (up/down) based on demand.
5. Deallocation: Releasing unused resources to save cost.

Resource Types

Type	Description	Examples
Compute	Processing units like CPUs, GPUs, or virtual machines	EC2, Azure VM, GCP Compute Engine
Storage	Persistent data storage systems	AWS S3, Azure Blob, Google Cloud Storage
Network	Connectivity and traffic routing	Elastic Load Balancer, VPC, VPN
Specialized	Optimized for specific workloads	AI accelerators, caching, FPGAs, CDN

Resource Management Layers

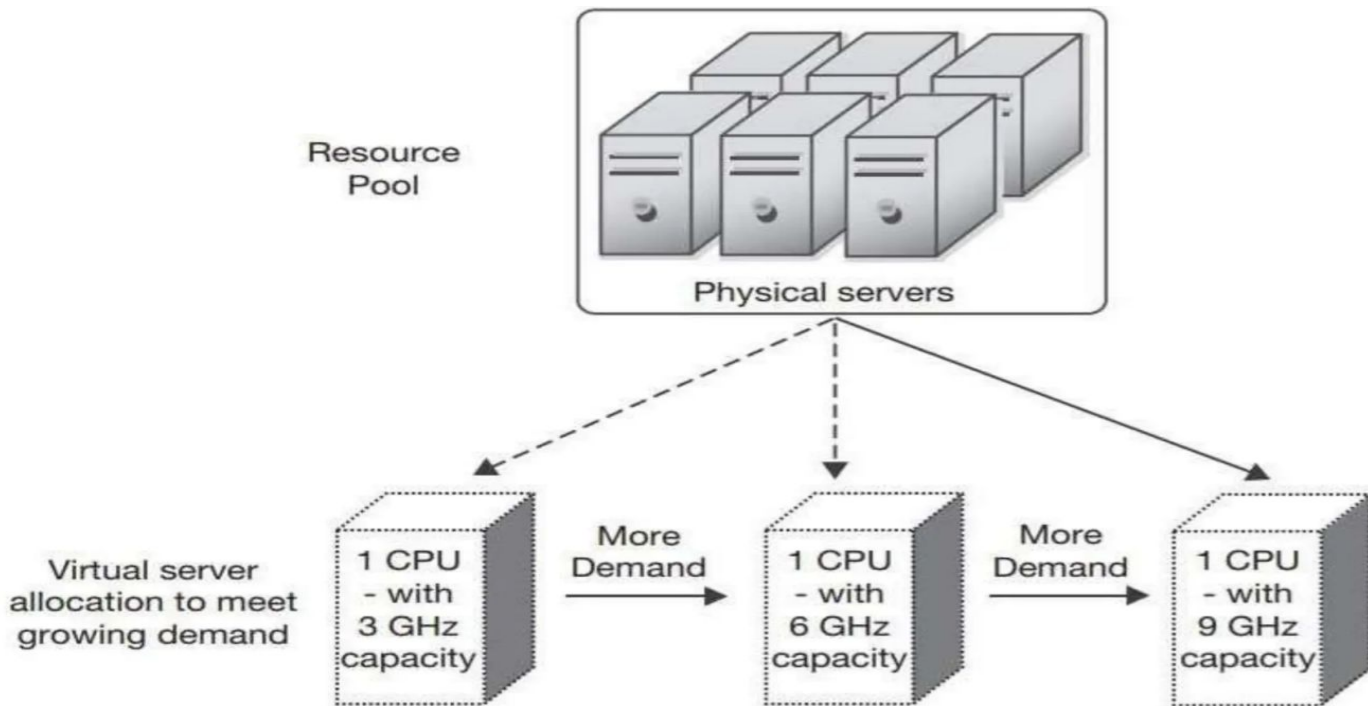
Layer	Description	Examples
Infrastructure Layer	Physical and virtualized hardware management	Hypervisors (VMware, KVM), Containers
Platform Layer	Middleware, orchestration, automation	Kubernetes, OpenStack, AWS Elastic Beanstalk
Application Layer	Policy-driven elasticity, cost optimization	Auto-scaling rules, service-level tuning

Scaling

- Scaling = Adjusting computing resources dynamically according to workload.
- **Purpose:**
 - Maintain performance during demand spikes.
 - Optimize resource usage during low demand.
- **Benefits:**
 - Cost-efficiency.
 - Consistent user experience.
 - Resource elasticity — grow or shrink as needed.
 - Continuous availability
- **Example:**
 - E-commerce site scales up during Black Friday traffic and scales down afterward.
- Supported across compute, storage, networking, and database services.

Types of Scaling

- **Vertical Scaling (Scale-Up):**
 - Add more power (CPU, RAM, storage) to an existing instance.
 - Limited by hardware capacity.
 - Simpler but can cause downtime.
 - *Example:* Upgrading t2.micro → t2.large on AWS EC2.
- **Horizontal Scaling (Scale-Out):**
 - Add more instances to handle increased load.
 - Offers redundancy and fault tolerance.
 - Requires load balancing.
 - *Example:* Adding multiple EC2 instances behind a load balancer.
- **Diagonal Scaling:**
 - Combination of vertical and horizontal scaling.
 - Add instances and upgrade capacity as needed.
 - Balances performance and cost.



Comparison: Vertical vs. Horizontal Scaling

Feature	Vertical Scaling	Horizontal Scaling
Complexity	Simple (single system)	Complex (requires orchestration)
Scalability Limit	Hardware-bound	Virtually unlimited
Downtime	Often required	Usually zero
Cost Efficiency	Expensive at high tiers	Better at large scale
Use Cases	Databases, legacy systems	Web apps, microservices

Scaling Strategies

1. Manual Scaling

- Admin manually adds or removes resources.
- Simple but slow response.
- Good for predictable workloads.

2. Automatic Scaling

A. Proactive Scaling

- Scheduled or predictive scaling
- Uses:
 - Historical workload patterns
 - Forecasting algorithms
- Example: Add VMs before peak business hours.

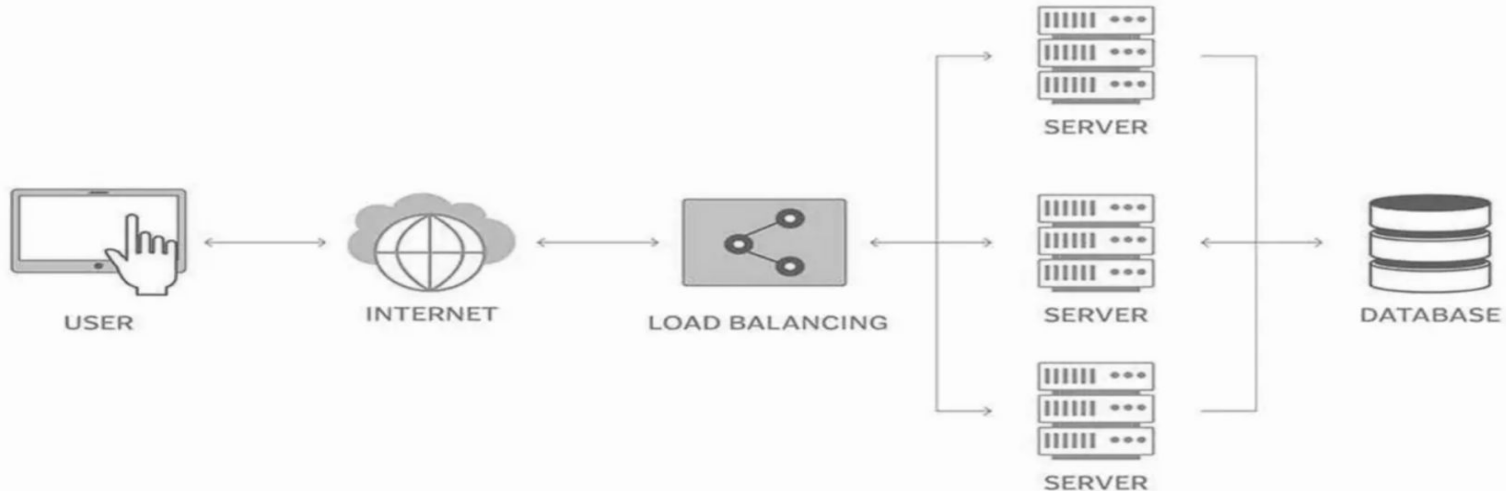
B. Reactive Scaling

- Trigger-based scaling
- Responds to:
 - High CPU, RAM
 - Queue length, traffic spikes
- Example: Add instances when CPU > 80%.

Load Balancing

- “Cloud load balancing” refers to distributing workloads and network traffic across multiple computing resources (servers, VMs, etc.) in a cloud environment.
- Instead of all traffic going to one server, a load balancer routes incoming traffic (“requests”) to different backend servers based on certain rules/policies.
- This helps in handling high user demands, avoiding overloading, and improving overall system responsiveness
- Helps support horizontal scaling by routing traffic to new instances.
- This prevents bottlenecks, improves performance, and maximizes resource utilization.
- Different strategies are used depending on system needs and workload behavior.

- Ensures:
 - Even workload distribution
 - No single point of failure
 - Faster response and elasticity



Types of Load Balancing

By technology:

- Hardware load balancer
- Software (or virtual) load balancer

By “type of service/request”:

- Application Load Balancing
- Network Load Balancing
- Global Server Load Balancing (GSLB)
- DNS Load Balancing

Hardware load balancer

- Dedicated physical appliance built solely for traffic distribution
- High performance: specialized chips (ASICs/FPGAs) handle large traffic volumes with very low latency
- Reliable & robust: enterprise-grade, often used in data centers
- Cons:
 - Expensive — hardware purchase + maintenance
 - Less flexible/scalable — scaling requires buying more physical units
 - Slower deployment compared to cloud-based options

Software (or virtual) load balancer

- Runs on standard servers or virtual machines
- Highly flexible — easy to deploy, update, or scale
- Cost-effective — no dedicated hardware needed
- Cloud-friendly — integrates well with auto-scaling and dynamic infrastructure
- Cons:
 - Performance depends on underlying hardware/VM resources
 - Might not match hardware LB throughput for massive enterprise traffic

Application Load Balancing

- Makes routing decisions using application-layer (Layer 7) data
- Uses details like:
 - HTTP headers
 - URLs / paths
 - Cookies
 - SSL session IDs
- Can route based on specific rules (e.g., `/api` → API servers, `/images` → static servers)
- Ideal for web applications, microservices, and content-based routing

Network Load Balancing

- Operates at the network layer (Layer 4)
- Makes decisions based on:
 - IP address
 - TCP/UDP ports
- Very fast and efficient; suitable for high-throughput, low-latency environments
- Works for any protocol, not just HTTP
- Common in real-time apps like gaming, VoIP, or streaming

Global Server Load Balancing (GSLB)

- Distributes traffic across servers in multiple geographic regions
- Chooses a server based on:
 - Client location
 - Regional server health/availability
 - Latency
 - Load/traffic conditions
- Improves global availability, disaster recovery, and user performance

DNS Load Balancing

- Uses DNS responses to distribute traffic
- A single domain (e.g., [example.com](#)) can resolve to different IPs
- Each DNS answer directs users to a different server or region
- Simple and scalable, but:
 - Not real-time (DNS caching delays)
 - Cannot detect server failure instantly

cloud load balancing techniques and algorithms

Static algorithms

- **Round Robin**
 - forwards requests to each cloud server in a simple, repeating order.
- **Weighted Round Robin**
 - assigns servers with greater capacity a higher “weight,” enabling them to receive more incoming application traffic.
- **IP Hash**
 - performs a hash, or a mathematical computation, on the client IP address, converting it to a number and mapping it to individual servers.

Dynamic algorithms

- **Least Connections**
 - distributes traffic to the servers with the fewest active connections.
- **Least Response Time**
 - directs traffic to the servers with the lowest average response time and fewest connections.
- **Least Bandwidth**
 - sends requests to the servers consuming the lowest amount of bandwidth during a recent period of time.

Cloud Load Balancing Examples

- **AWS Elastic Load Balancer (ELB):**
 - Types: Application, Network, and Gateway Load Balancers.
 - Integrated with EC2 Auto Scaling Groups.
- **Google Cloud Load Balancer:**
 - Global scale, anycast IP routing, multi-region balancing.
- **Azure Application Gateway:**
 - Layer 7 routing, URL-based rules, WAF integration.
- **Other Tools:** NGINX, HAProxy, Cloudflare Load Balancer.

Redundancy

- Redundancy is the duplication of critical components or systems with the intent of increasing reliability and availability.
- **Purpose:**
 - Ensures continuous operation even if a component fails.
 - Provides fault tolerance and business continuity.
- **Implemented at multiple layers:**
 - Hardware (servers, storage)
 - Software (clusters, containers)
 - Data (replication and backups)

Types of Redundancy

1. **Hardware Redundancy**

- Duplicate physical components: servers, power supplies, network switches.
- Example: RAID (Redundant Array of Independent Disks) for disk mirroring.
- Hot-swappable power supplies for servers.

2. **Software Redundancy**

- Multiple software modules performing similar tasks for backup.
- Examples: Active-Passive clustering, load-balanced microservices.

3. **Data Redundancy**

- Data replicated across multiple storage systems or availability zones.
- Cloud examples: AWS S3 Multi-AZ replication, Google Cloud Storage replication.

Strategies for Redundancy

1. Replication

- Deploys multiple copies of data, applications, and services across different servers or regions.
- Multi-region replication increases resilience—if one geographic location experiences an outage, the replicated region continues operating.
- Helps protect against data loss and minimizes service downtime.

2. Redundant Infrastructure

- Cloud providers maintain duplicate hardware and network components across availability zones or regions.
- Redundant components include:
 - Compute servers
 - Network devices
 - Storage systems
 - Power supplies
- If any component fails, operations automatically shift to the backup infrastructure.

3. Failover Mechanisms

- Automated failover systems detect resource failures and immediately redirect workloads to standby systems.
- Ensures seamless continuity without manual intervention.
- Commonly used with:
 - Databases
 - Load balancers
 - Application clusters

4. Load Balancing for Redundancy

- Traffic is distributed across multiple active servers or instance pools.
- If one server becomes unavailable:
 - Remaining servers continue serving traffic
 - Users experience minimal disruption
- Supports both performance and resilience.

5. Monitoring & Automation

- Continuous monitoring tools track:
 - System health
 - Resource usage
 - Application performance
- Automated scripts or orchestration tools:
 - Trigger failover
 - Restart unhealthy resources
 - Provision additional instances if needed
- Enhances reliability with real-time response to issues.

High Availability (HA)

- HA systems are designed to operate continuously without interruption for a long period of time.
- Ensures users have access to resources at all times.
- **Goal:** Minimize downtime due to hardware/software failures or maintenance.
- **Uptime Measurement:**
 - Expressed in “**nines**”:
 - 99% → ~3.65 days downtime/year
 - 99.9% → ~8.7 hours
 - 99.99% → ~52 minutes
 - 99.999% → ~5 minutes (“Five Nines”)
- **Key Components:**
 - Redundancy
 - Load balancing
 - Failover mechanisms

Fault Tolerance vs High Availability

Aspect	Fault Tolerance	High Availability
Reaction	Immediate recovery; system continues seamlessly.	Quick failover to standby components.
Objective	Zero interruption.	Minimal downtime.
Cost	High, due to duplication of all components.	Moderate, relies on redundancy and failover.
Complexity	Requires real-time synchronization.	Easier with automation and clustering.
Example	RAID 1 mirroring, ECC memory.	Active-passive cluster, load balancer.

Building blocks of high availability

- **Redundancy**

- Duplicate servers, storage, network, even data-centers — if one fails, another continues serving.
- This avoids single points of failure.

- **Monitoring & Failure Detection**

- Continuously monitors system health (CPU, memory, I/O, etc.) to spot failure or degradation early.
- Enables proactive response.

- **Failover Mechanisms**

- When a component fails, automatically switch operations to backup resources (automatic failover) — reduces downtime.

- **Load Balancing & Clustering**

- Spread traffic across multiple servers or cluster nodes; if one node fails, others handle load.
- Enhances performance + availability.

- **Data Backup and Recovery**

- Regular backups (local or remote/cloud) to enable data and application recovery if failure or disaster strikes.

Data Replication in Cloud

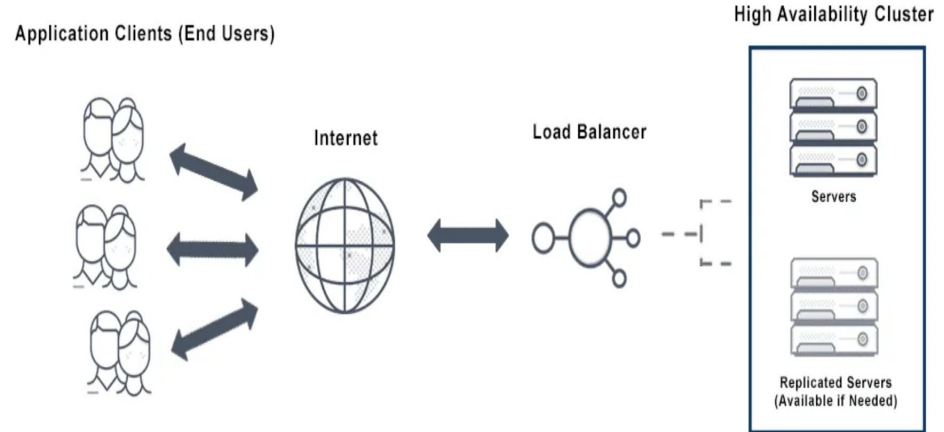
- **Purpose:**
 - Protects against data loss and ensures durability.
- **Replication Models:**
 - **Synchronous Replication:**
 - Data written to both primary and replica simultaneously.
 - Guarantees consistency but adds latency.
 - Best for databases.
 - **Asynchronous Replication:**
 - Primary acknowledges write first; replica updated later.
 - Provides better performance; eventual consistency.
- **Example:**
 - AWS S3 Cross-Region Replication, Google Cloud Dual-Region Storage.

Disaster Recovery (DR)

- A structured approach to recover IT systems after major failures (natural disaster, cyber attack).
- **Objectives:**
 - **RTO (Recovery Time Objective):**
 - Maximum downtime tolerated.
 - **RPO (Recovery Point Objective):**
 - Maximum data loss acceptable.
- **DR Strategies:**
 - Backup and restore.
 - Pilot light (minimal environment kept ready).
 - Warm standby (scaled-down replica).
 - Multi-site (active-active).
- **Cloud Services for DR:**
AWS Backup, Azure Site Recovery, Google Cloud Backup.

High Availability Architecture Example

- **Active-Active Architecture:**
 - Multiple servers or regions serve requests simultaneously.
 - Load balanced, redundant databases, and failover routing.
- **Components:**
 - Load balancer
 - Application servers in multiple zone
 - Replicated database clusters
 - Global DNS routing
- **Benefits:**
 - Zero downtime deployment
 - Automatic traffic redirection
 - Resilience against regional failures



HA-Relevant Metrics & Concepts

To measure and design HA systems, some standard metrics/concepts are used

- **Uptime (%)** — proportion of time system is operational (e.g. 99.9%, 99.99%, 99.999%).
- **Recovery Time Objective (RTO)** — how long the system can be down before causing unacceptable impact.
- **Recovery Point Objective (RPO)** — how much data (in time) an outage can lose (i.e. how recent backups should be).
- **Mean Time Between Failures (MTBF) & Mean Time To Recovery (MTTR)** — how often failures happen, and how fast system recovers.

Cloud Monitoring

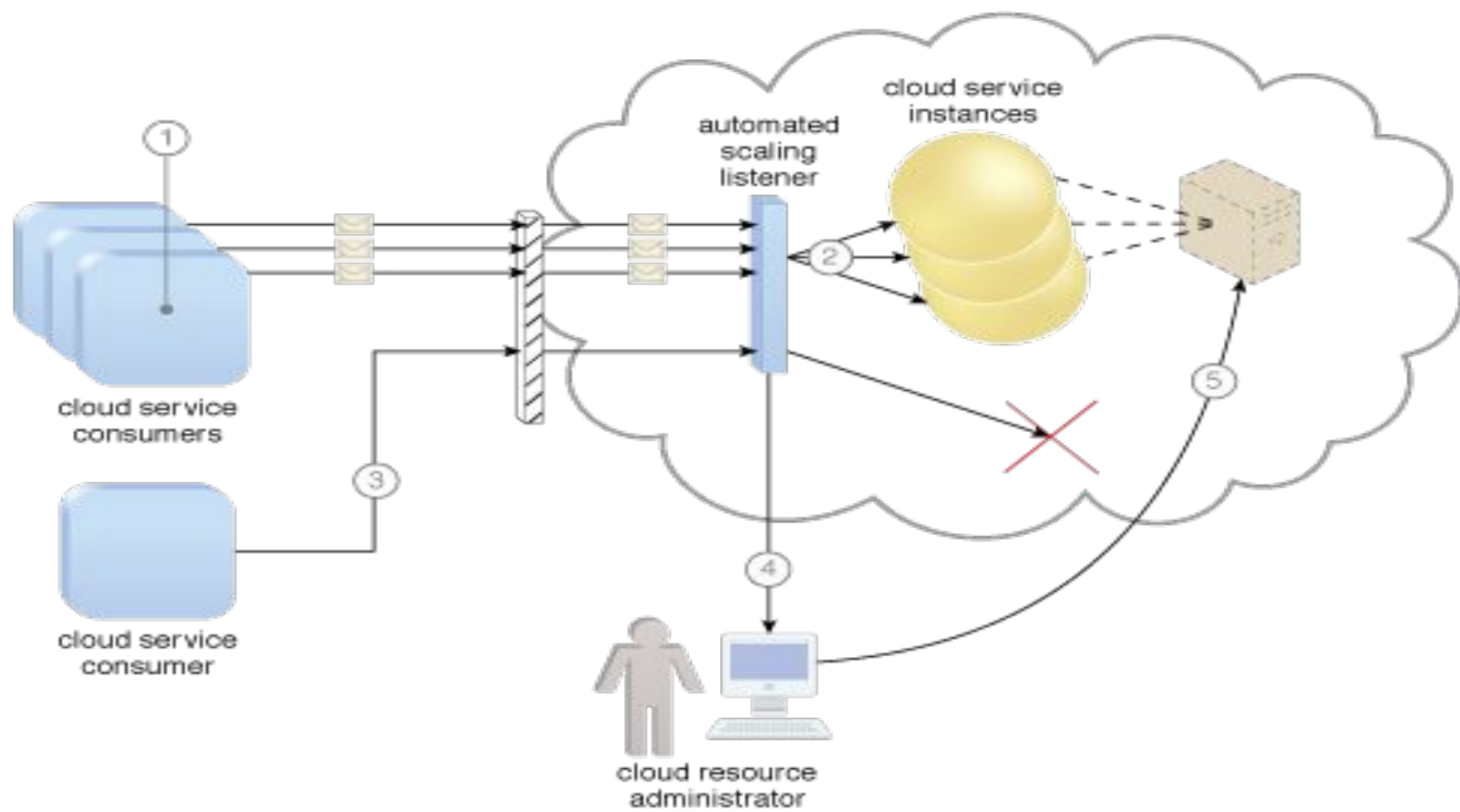
- Cloud monitoring involves continuously observing, analyzing, and managing the performance, availability, and security of cloud resources.
- **Key Goals:**
 - **Detect issues proactively:** Identify performance degradation or failures before they impact users.
 - **Optimize performance and cost:** Analyze usage metrics to right-size resources.
 - **Ensure SLA compliance:** Meet service uptime, latency, and reliability commitments.
 - **Support automated actions:** Enable triggers for scaling, alerts, or fault recovery.
- **Commonly Monitored Metrics:**
 - CPU utilization, memory, network throughput, storage I/O, latency, error rates.

Cloud Usage Monitor

- Tracks and records the utilization of cloud resources (compute, storage, and network).
- **Purpose:**
 - Identify overused or underutilized resources.
 - Provide insights for scaling and cost optimization.
 - Generate reports for billing and capacity planning.
- **Metrics Captured:**
 - CPU, disk, memory, network bandwidth, I/O requests.
- **Examples:**
 - **AWS CloudWatch:** Monitors EC2, S3, and RDS metrics.
 - **Azure Monitor:** Collects telemetry across applications and infrastructure.
 - **Google Cloud Operations Suite (formerly Stackdriver).**

Automated Scaling Listener

- A monitoring component that listens to metric thresholds and triggers scaling actions automatically.
- **Functionality:**
 - Continuously observes parameters like CPU load, network latency, or queue length.
 - When threshold exceeded → **Scale-Out**.
 - When usage drops → **Scale-In**.
- **Key Components:**
 - **Metrics source:** CloudWatch, Azure Metrics, Prometheus.
 - **Listener logic:** Compares metrics to scaling policies.
 - **Action engine:** Launches or terminates resources.
- **Example:**
 - AWS Auto Scaling with target tracking policy: *Maintain average CPU utilization = 60%.*

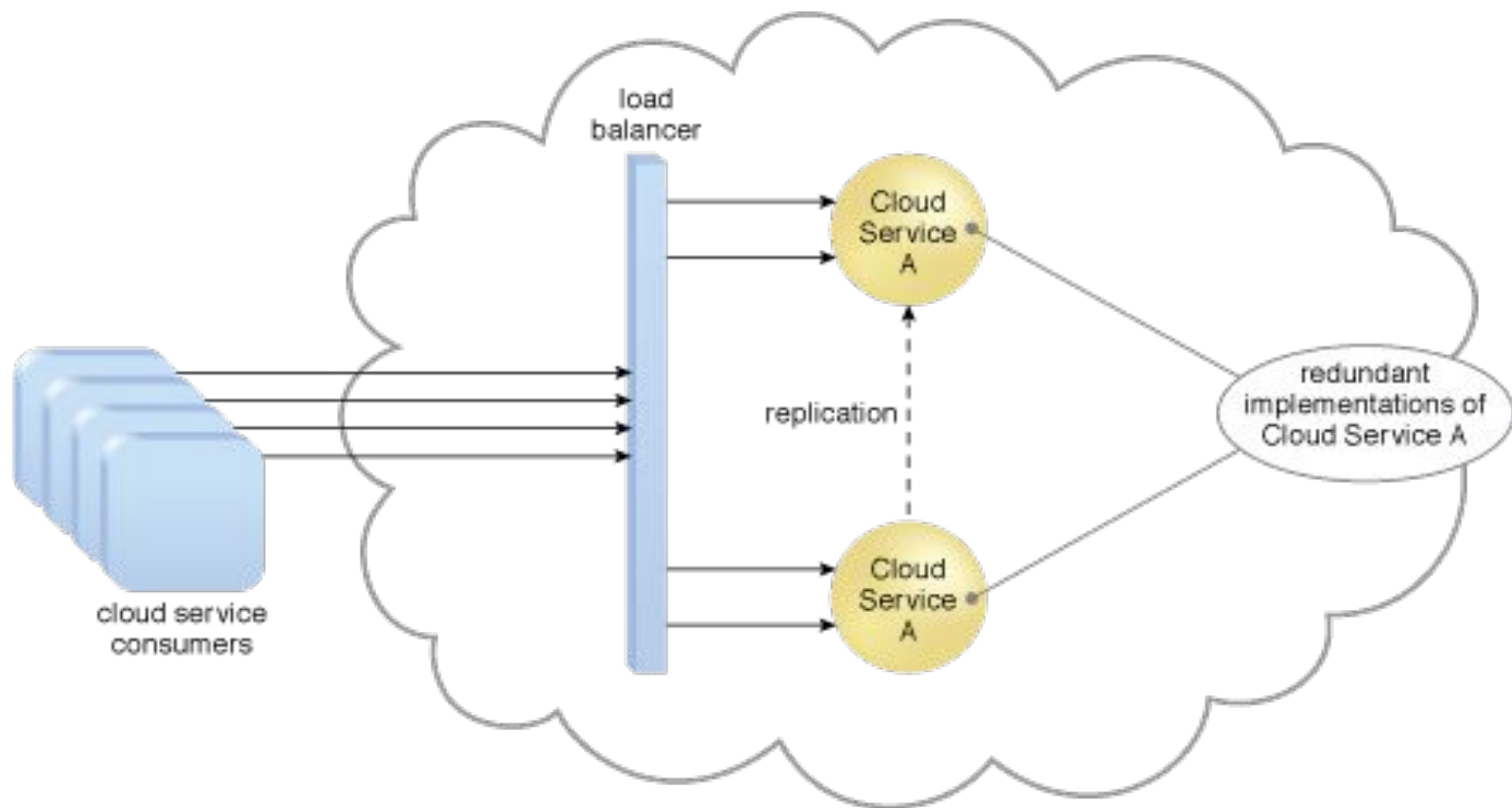


Process Flow:

1. Cloud service consumers send multiple service request messages to access the cloud service.
2. The automated scaling listener receives the incoming requests and forwards them to available cloud service instances while monitoring workload conditions.
3. Additional requests from cloud service consumers may enter the system through alternate request paths, increasing the load on the cloud service.
4. The cloud resource administrator monitors the automated scaling process and manages scaling policies and resource allocation when required.
5. When increased demand or failure is detected, new cloud service instances are provisioned automatically, and failed instances are excluded from request handling.

Load Balancing Monitor

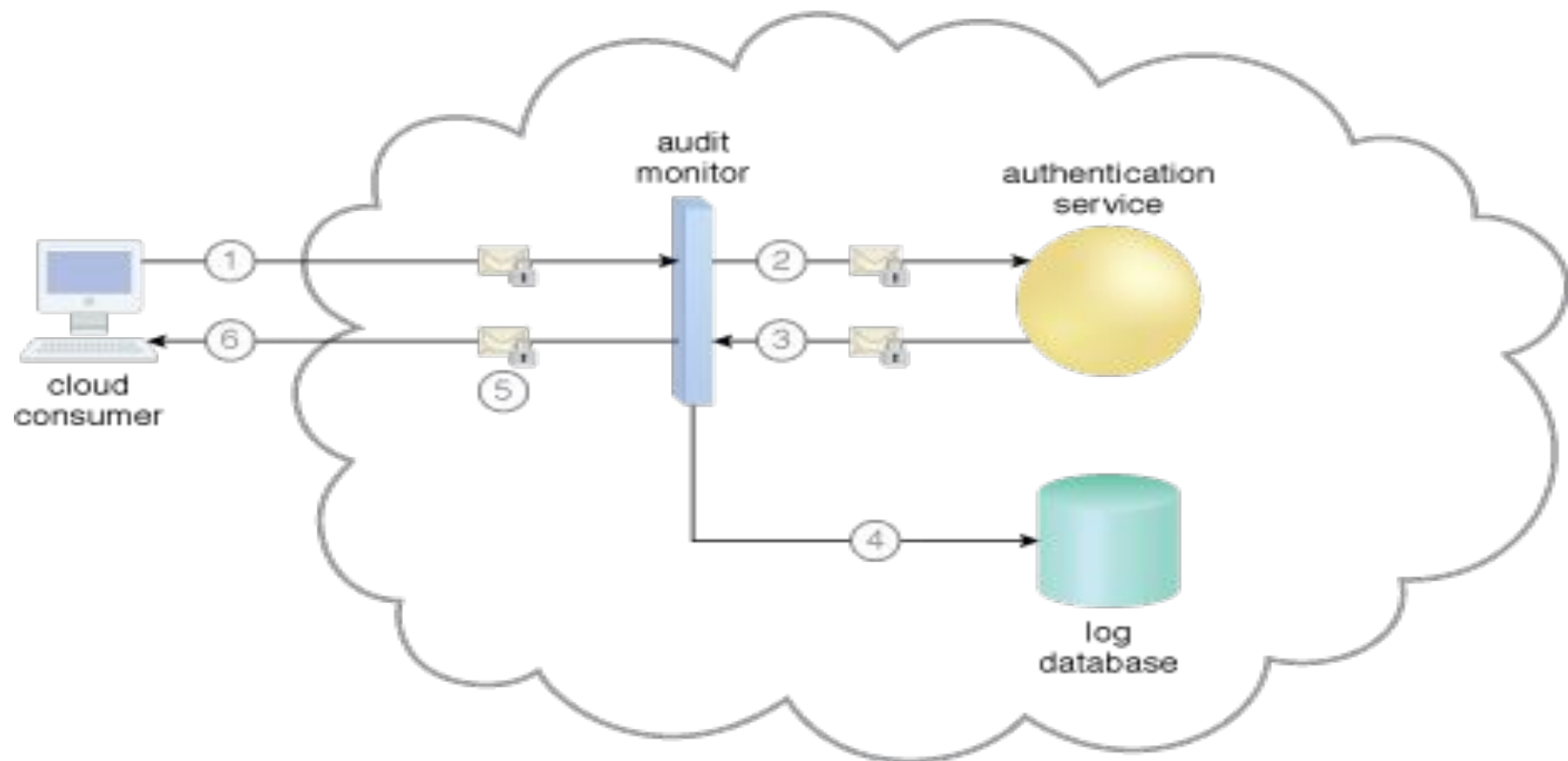
- A mechanism that observes load distribution across servers or instances and adjusts routing in real time.
- **Key Features:**
 - Monitors request rate, connection count, response latency, and error rate.
 - Automatically detects unhealthy instances and reroutes traffic.
 - Supports dynamic scaling integration.
- **Common Techniques:**
 - Health checks and weighted algorithms.
 - Adaptive balancing based on real-time metrics.
- **Examples:**
 - AWS Elastic Load Balancer (ELB), Azure Application Gateway, GCP Load Balancer.



1. Cloud service consumers send service request messages to access Cloud Service A.
2. The load balancer receives all incoming request messages from the cloud service consumers.
3. The load balancer distributes the incoming requests evenly among multiple instances of Cloud Service A.
4. Replication occurs between the Cloud Service A instances to synchronize data and maintain consistency.
5. Redundant implementations of Cloud Service A ensure that if one instance fails, requests are redirected to another active instance, and responses are returned to the consumers.

Audit Monitor

- Tracks all user activities, configuration changes, and API calls for accountability, compliance, and security.
- **Functions:**
 - Records who did what, when, and from where.
 - Detects unauthorized or suspicious access.
 - Supports forensic analysis after incidents.
- **Examples:**
 - **AWS CloudTrail:** Logs every API call (create, modify, delete).
 - **GCP Cloud Audit Logs:** Monitors admin and data access events.
 - **Azure Activity Logs.**
- **Use Cases:**
 - Compliance (GDPR, HIPAA).
 - Security auditing.
 - Intrusion detection.



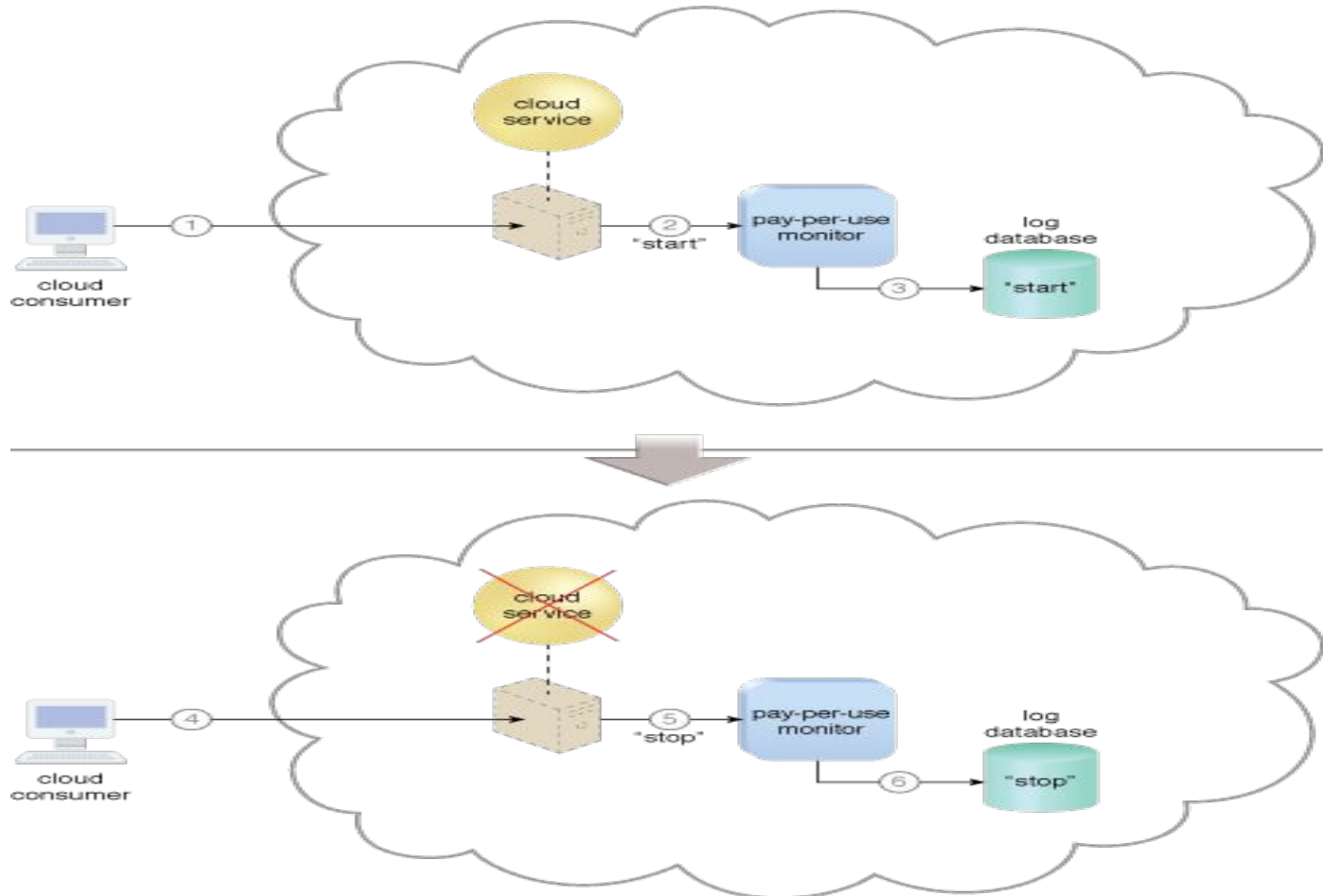
Process Flow:

1. A cloud service consumer requests access to a cloud service by sending a login request message with security credentials.
2. The audit monitor intercepts the message.
3. And forwards it to the authentication service.
4. The authentication service processes the security credentials. A response message is generated for the cloud service consumer, in addition to the results from the login attempt.
5. The audit monitor intercepts the response message and stores the entire collected login event details in the log database, as per the organization's audit policy requirements.
6. Access has been granted, and a response is sent back to the cloud service consumer.

Pay-Per-Use Monitor

- The pay-per-use monitor mechanism measures cloud-based IT resource usage in accordance with predefined pricing parameters and generates usage logs for fee calculations and billing purposes.
- Some typical monitoring variables are:
 - request/response message quantity
 - transmitted data volume
 - bandwidth consumption
- The data collected by the pay-per-use monitor is processed by a billing management system that calculates the payment fees.

- **Functionality:**
 - Tracks compute hours, storage GB-months, data transfer, API calls.
 - Provides real-time cost estimation.
 - Supports chargeback and showback models in organizations.
- **Benefits:**
 - Transparent billing.
 - Cost control and forecasting.
 - Integration with monitoring for anomaly detection.
- **Examples:**
 - AWS Cost Explorer & Billing Dashboard.
 - Azure Cost Management.
 - GCP Billing Reports.



Process Flow

1. A cloud consumer requests the creation of a new instance of a cloud service
2. The IT resource is instantiated and the pay-per-use monitor mechanism receives a “start” event notification from the resource software
3. The pay-per-use monitor stores the value timestamp in the log database
4. The cloud consumer later requests that the cloud service instance be stopped
5. The pay-per-use monitor receives a “stop” event notification from the resource software and stores the value timestamp in the log database

Monitoring Tools

- **Open-Source Tools:**
 - **Prometheus:** Time-series metrics collection.
 - **Grafana:** Visualization dashboards.
 - **Zabbix / Nagios:** Infrastructure health checks.
- **Commercial / Cloud-Native Tools:**
 - AWS CloudWatch, Azure Monitor, Datadog, Splunk.
- **Features:**
 - Custom dashboards and alerting.
 - Log aggregation and analysis.
 - Anomaly detection via ML.
 - Integration with ticketing and notification systems.

Service Level Agreement (SLA)

- A formal agreement between a cloud service provider and a customer that defines the expected performance and quality of service.
- Today, most cloud SLAs are standardized, especially for large providers.
- **Purpose:**
 - Establishes clear performance standards.
 - Ensures accountability between both parties.
 - Forms a legal and operational framework for service delivery.
- What an SLA usually specifies:
 - Service availability (uptime)
 - Response time / latency
 - Reliability of service components
 - Responsibilities of each party
 - Warranties and penalties

- **Types of SLAs:**
 - **Customer-based SLA:**
 - Agreement between provider and individual customer.
 - Covers all services used by that customer i.e Customized for a specific customer.
 - Example: A company using AWS for compute + storage under one SLA.
 - **Service-based SLA:**
 - Covers a specific service (e.g., Compute SLA, Database SLA).
 - Same SLA for all users of a service.
 - Focused on service performance metrics.
 - **Multi-level SLA:**
 - Combines multiple agreements (corporate, customer, service).
 - Different SLA levels for different customers or services.
 - Allows customization for departments or regions.

SLA Parameters

- **Common SLA Metrics:**
 1. **Availability / Uptime:**
 - Example: *99.9% availability* → ~8.7 hours downtime per year.
 2. **Performance:**
 - Response time, throughput, latency, transaction rate.
 3. **Support Response Time:**
 - Maximum time allowed for customer support to respond to incidents.
 4. **Security & Compliance:**
 - Data encryption, privacy adherence (GDPR, HIPAA).
 5. **Reliability:**
 - Mean Time Between Failures (MTBF) and Mean Time To Repair (MTTR).

SLA Violations

- If the provider fails to meet SLA targets, they must compensate the customer, similar to an insurance policy.
- When SLA is Breached:
 - Provider fails to meet committed performance (e.g., uptime < 99.9%).
- Common Remedies:
 - Service Credits: Financial compensation applied to next billing cycle.
 - Refunds: For severe or repeated violations.
- Proactive Monitoring:
 - Alerts triggered before SLA breach (predictive analytics).
- Reporting:
 - Transparent reporting ensures customer confidence.

Example:

AWS gives a 10% service credit if EC2 uptime < 99.99%.

Billing Management System

- Tracks, measures, and invoices resource usage.
- Enables pay-as-you-go pricing models.
- **Components:**
 - Usage Metering:
 - Monitors actual resource consumption.
 - Pricing Engine:
 - Applies rates to usage data.
 - Billing & Invoicing Module:
 - Generates bills automatically.
 - Customer Portal:
 - Provides cost visibility and reports.
- **Integration:**
 - Connected with monitoring systems for accurate real-time billing.

Cloud Billing Models

Model	Description	Example
On-Demand	Pay for actual usage (no commitment).	AWS EC2 on-demand.
Reserved Instances	Long-term commitment (1–3 years) for discount.	Azure Reserved VM.
Spot Instances	Use unused capacity at variable pricing.	AWS Spot Instances.
Subscription-based	Fixed monthly/yearly cost for predictable workload.	SaaS model (e.g., Office 365).

Cost Optimization Techniques

- Rightsizing:
 - Choose correct instance size and type. Avoid overprovisioning.
- Auto Scaling:
 - Scale resources up/down based on demand.
- Storage Lifecycle Policies:
 - Move infrequently accessed data to cheaper storage (e.g., Glacier).
- Reserved/Spot Instances:
 - Combine for predictable + flexible workloads.
- Monitoring Idle Resources:
 - Shut down unused VMs, orphaned disks, and unattached IPs.
- Budget Alerts:
 - Set spending limits and receive notifications.

Case Study: AWS Cost Explorer & Budgets

- Scenario:
A company notices rising cloud costs due to unmonitored EC2 instances.
- Solution:
 - Use AWS Cost Explorer to visualize cost trends.
 - Create Budgets for compute, storage, and data transfer.
 - Set email/SNS alerts for exceeding thresholds.
- Outcome:
 - Reduced cost overruns by 25%.
 - Improved forecasting accuracy.

Challenges in Cloud Resource Management

1. Multi-objective Goals:

- Trade-offs between performance, cost, and energy.
- SLAs vs. cost savings.

2. Heterogeneity:

- Managing VMs, containers, serverless functions.
- Varying user workloads and application architectures.

3. Decentralization:

- No global system state.
- Cloud resources span multiple regions and zones.

4. Elastic Demand:

- Workloads can spike unpredictably.
- Provisioning too much leads to waste; too little causes performance degradation.

Thank you